

Docker for Robotics

Intelligent Robotics I

Andrés Suárez García

University of Vigo

Resources	2
What is Docker?	4
Installation	8
Usage	12
Dockerfile & Docker Compose	19
Docker for Robotics	25

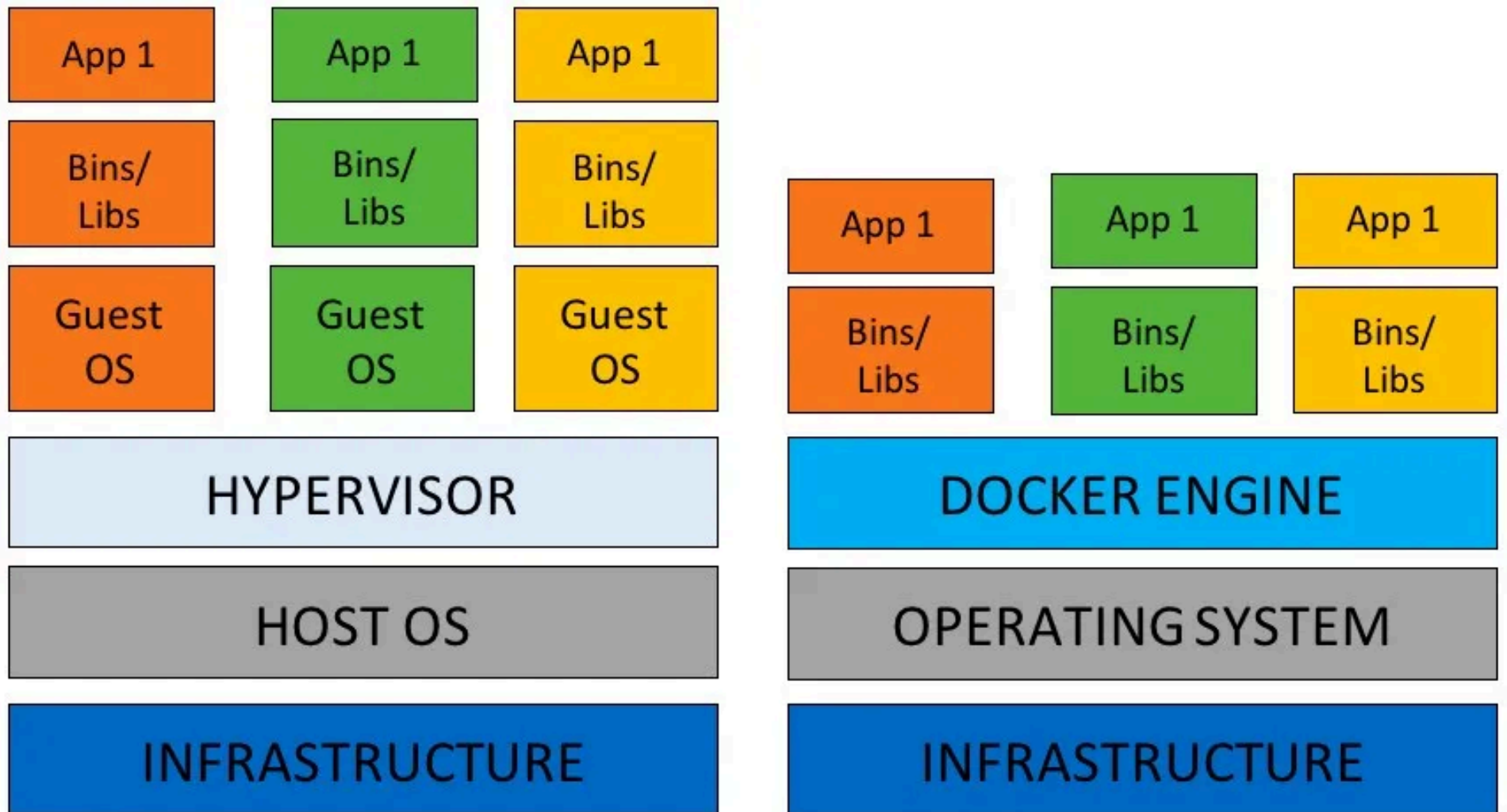
Resources

This presentation is based on the following resources:

- “Docker for Robotics” by Articulated Robotics / [link](#)
- “Docker for Robotics with The Operating System” by Tobit Flatscher / [link](#)

What is Docker?

What is Containerization?

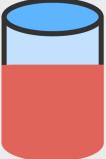
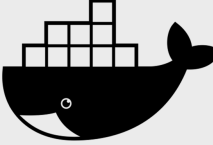
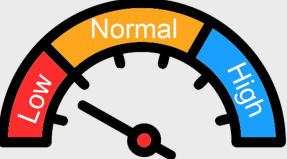
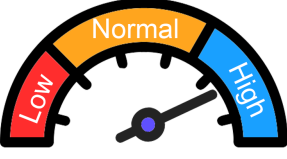


Source

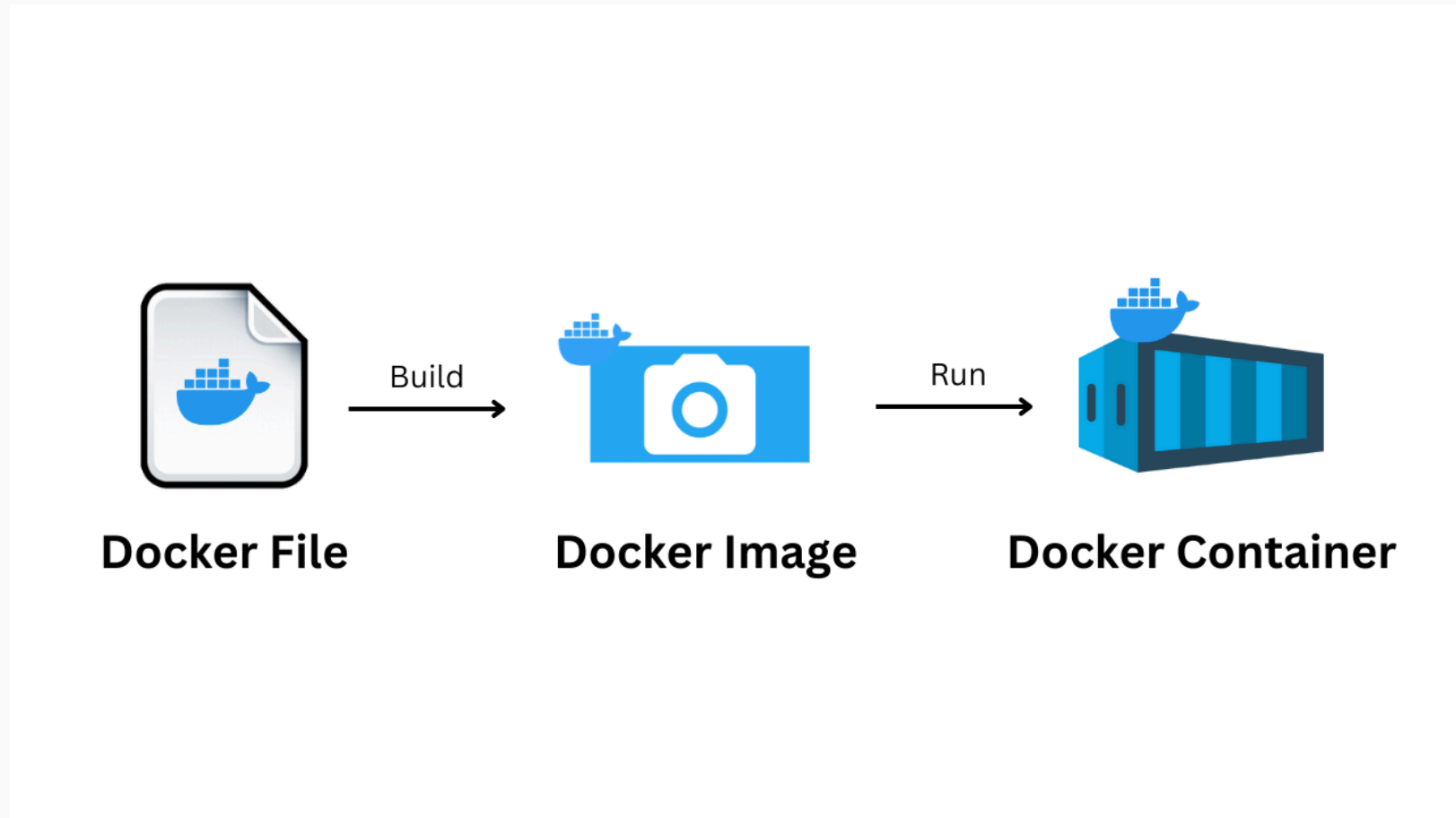
Why?



blog.bytebytego.com

	Virtualization	Containerization
Startup time	 minutes	 seconds
Disk space		
Portability	Less Portable	
Efficiency		
Operating system/kernel	Dedicated	Shared

An image is a blueprint of a container.



Source

Installation



Option 1: using the script in Docker webpage

```
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh ./get-docker.sh --dry-run
```

Option 2: using Snap package manager

```
sudo apt update  
sudo apt install snapd  
sudo snap install docker
```

Check if docker is correctly installed:

```
sudo systemctl is-enabled docker  
sudo docker run hello-world
```

In order to not use sudo with docker:

```
sudo groupadd docker  
sudo usermod -aG docker $USER
```

Installation WSL

Actually, using Snap package in WSL gives problems with GUI apps. Hence, the Docker script launches a warning and stops the installation. The following steps are recommended.

```
# Update package list and install prerequisites
```

```
sudo apt-get update
```

```
sudo apt-get -y install ca-certificates curl gnupg
```

```
# Add Docker's GPG key
```

```
sudo mkdir -p /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
# Add Docker repository
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/  
docker.gpg]
```

```
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \  
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
# Update package list and install docker apps
```

```
sudo apt-get update
```

```
sudo apt-get -y install docker-ce docker-ce-cli containerd.io
```

```
sudo apt-get -y install docker-compose-plugin
```

Usage



```
docker <base_command> <child_command> <arguments>
```

where:

- <base_command> can be 'image' or 'container'
- <child_command> is a command depends on <base_command>
- All commands has a short version

Command 'image'

list

`docker image ls`

`docker images`

pull (downloads image)

`docker image pull <image_name>`

`docker pull <image_name>`

build (build image)

`docker image build <image_name>`

`docker build <image_name>`

delete

`docker image rm <image_id>`

`docker rmi <image_id>`

prune (delete all unused images)

`docker image prune`

Command 'container' 1/

run (makes a container from an image)

```
docker container run <image_name>
```

run advanced

-i: interactive

-t: terminal

-rm: delete the container after running

```
docker run -it --rm <image_name>
```

list (-a all containers)

```
docker container ls -a
```

```
docker ps -a
```

stop (stop a container before delete it)

```
docker container stop <container_name>
```

```
docker stop <container_name>
```


start

`docker container start container_name`

`docker start container_name`

delete

`docker container rm container_name`

`docker rm container_name`

delete all containers

`docker container prune`

exec program inside a running container

`docker exec -it container_name program`

`docker exec -it goofy_allen /bin/bash`

Difference between run and start

- run command creates a new container

```
docker run <container_name>
```

- start continues with the container state

```
docker start -i <container_name>
```

```
docker run -it \  
-v absolute_path_host:absolute_path_container \  
<image_name>
```

- volumes are host directories mounted like virtual drives inside containers
- changes will be stored in the host directory
- they can be shared by several containers at the same time

Dockerfile & Docker Compose

- a Dockerfile is a text file
- helps to make projects easily reproducible
- it has instructions to build a Docker image
- Dockerfile and the files required must be in the same folder

```
# build image  
docker build -t my_image .
```

```
# run container  
docker run -it my_image
```

Dockerfile Example for ROS2

```
# Use ROS 2 Humble as base
FROM osrf/ros:humble-desktop-full

# Set working directory
WORKDIR /ros_ws

# Copy your ROS 2 workspace
COPY . .

# Install dependencies and build
RUN apt-get update && rosdep install --from-paths src --ignore-src -r -y
RUN colcon build

# Source the workspace and run a ROS 2 node
CMD ["bash", "-c", "source install/setup.bash && ros2 run my_package my_node"]
```

- it uses a docker-compose.yml file
- this file describes how containers work together
- run all containers with one command

```
# turns on  
docker-compose up
```

```
# turns off  
docker-compose down
```

Compose Example for ROS2

```
version: '3.9'

services:
# runs ros2 nodes and launch files
ros-core:
  image: osrf/ros:humble-desktop-full
  volumes:
    - ./ros_ws:/ros_ws
  working_dir: /ros_ws
  command: bash -c "source /ros_ws/install/setup.bash && ros2 launch
my_package my_launch_file.launch.py"

# runs Gazebo simulator for robotics
gazebo:
  image: osrf/ros:humble-desktop-full
  environment:
    - DISPLAY=${DISPLAY}
  volumes:
    - /tmp/.X11-unix:/tmp/.X11-unix
  command: bash -c "gazebo"
```


Key Differences

Dockerfile	Docker Compose
Builds one image	Manages multiple images
Use for workspace setp	Nodes, Simulator, etc
Syntax: Dockerfile	Sytanx: YAML

Docker for Robotics

```
# -it, runs container in interactive mode with a terminal
# --rm, delete container after use (optional)
# --env='DISPLAY', passes host display to container
# --env='QT_X11_NO_MITSHM=1', prevents problems with Qt apps
# --volume, mounts X11 socket for GUI rendering from the host
```

```
xhost +local:docker && \
docker run -it --rm \
--env="DISPLAY" \
--env="QT_X11_NO_MITSHM=1" \
--volume="/tmp/.X11-unix:/tmp/.X11-unix:rw" \
osrf/ros:humble-desktop-full
```

```
--gpu all  
--runtime=nvidia  
--env="QT_X11_NO_MITSHM=1"  
--env="NVIDIA_DRIVER_CAPABILITIES=all"  
--env="NVIDIA_VISIBLE_DEVICES=all"  
--device=/dev/dri:/dev/dri
```

Learn more about Nvidia configuration / [link](#)